

Base de données NoSQL

Au-delà du modèle non relationnel

Louis Roussel

louis.roussel@univ-lille.fr

Support de cours par **Bastien Cazaux** qui s'appuie sur les cours d'**Anne Etien** et de **Anne Cécile Caron** de l'université de Lille et celui de **Régis Behmo** et **Nicolas Travers** de CentralSupélec.

Plan du cours

Plan du cours
Limites de NoSQL et nouveaux besoins
Bases de données multi-modèles

Bases de données NewSQL
Bases de données temporelles
Bases de données vectorielles

Constat : NoSQL a répondu à des problèmes de passage à l'échelle et de flexibilité, mais montre aujourd'hui certaines limites.

Limites de NoSQL

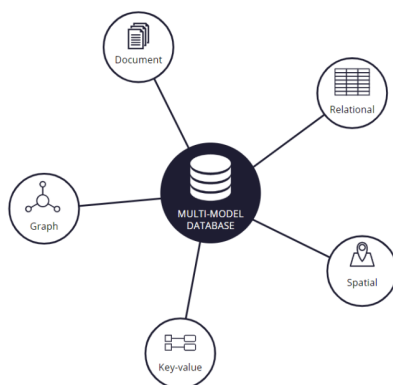
- Faible support des transactions complexes (ACID)
- Modèles parfois trop spécialisés ou peu expressifs
- Requêtes analytiques et jointures limitées
- Difficulté à gérer certaines données structurées ou temporelles

Besoins actuels

- Unifier plusieurs modèles dans un même système
- Concilier **scalabilité** et **cohérence forte**
- Gérer des données **complexes** (temps, vecteurs, graphes)
- Supporter des **usages émergents** (IA, recherche sémantique)

Bases de données multi-modèles

Idée clé : Une base de données multi-modèle supporte **plusieurs modèles de données** au sein d'un même moteur et d'un même système de requêtes.



Pourquoi le multi-modèle ?

- Éviter la multiplication des bases spécialisées
- Adapter le modèle aux usages (document, graphe, clé-valeur)
- Réduire la complexité architecturale des systèmes
- Faciliter l'évolution des applications

Exemples de bases de données multi-modèles

- **OrientDB, ArangoDB et SurrealDB** : document + graphe + clé-valeur
- **Azure Cosmos DB** : document + graphe + clé-valeur + colonne (via APIs SQL, Mongo, Cassandra, Gremlin)

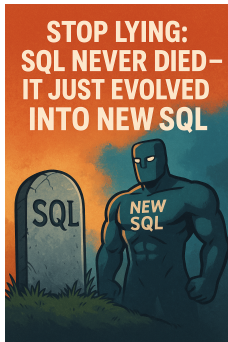
source : <https://linkurious.com/blog/visualize-cosmos-db-graph-linkurious/>

Idée clé : Les bases de données **NewSQL** cherchent à combiner :

- la **scalabilité horizontale** de NoSQL
- avec les **garanties ACID** et le **modèle relationnel** de SQL

Pourquoi NewSQL ?

- NoSQL sacrifie souvent la cohérence forte
- Les bases relationnelles classiques passent mal à l'échelle
- Certaines applications exigent **transactions + performance + distribution**



Caractéristiques principales

- Modèle relationnel et langage SQL
- Transactions ACID distribuées
- Architecture distribuée (partitionnement, réplication)
- Optimisation pour les charges transactionnelles (OLTP)

Exemples de bases NewSQL : CockroachDB, Google Spanner, TiDB, ...

source : <https://medium.com/@BuildShift/stop-lying-sql-never-died-it-just-evolved-into-newsql-dbf66ab6628>

Bases de données temporelles

Idée clé : Les bases de données temporelles sont conçues pour stocker et interroger des données qui évoluent dans le temps, en conservant l'historique des valeurs plutôt que seulement l'état courant.

Pourquoi ont-elles été créées ? Les SGBD classiques ont plusieurs limites face aux données temporelles :

- Ils écrasent souvent les anciennes valeurs (perte d'historique)
- Les requêtes sur le passé sont complexes et coûteuses
- La gestion du temps est laissée à l'application

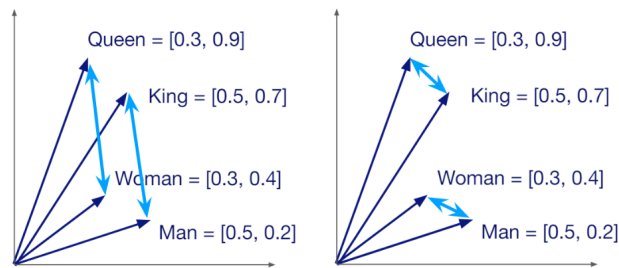
Or, de nombreux domaines en ont un besoin critique :

- Capteurs, IoT, télémétrie
- Finance (cours, transactions)
- Logs, monitoring, observabilité
- Audit, traçabilité, conformité

Exemples de bases NewSQL :

- **InfluxDB** : Très utilisée pour IoT, monitoring, métriques
- **TimescaleDB** : Extension PostgreSQL pour données temporelles
- **QuestDB** : Haute performance, orientée temps réel
- **Prometheus** : Monitoring d'infrastructure

Idée clé : Une base de données vectorielle stocke des données sous forme de **vecteurs numériques** et permet de rechercher des éléments **similaires** plutôt qu'identiques.



Caractéristiques principales

- Stockage de vecteurs de grande dimension
- Recherche par similarité (k-NN, distance cosinus, euclidienne)
- Index spécialisés (ANN : Approximate Nearest Neighbors)
- Intégration fréquente avec l'IA et le NLP

Exemples de bases vectorielles

- **Pinecone** : référence industrielle / cloud
- **Weaviate** : open-source + concepts bien exposables
- **Milvus** : performance et échelle